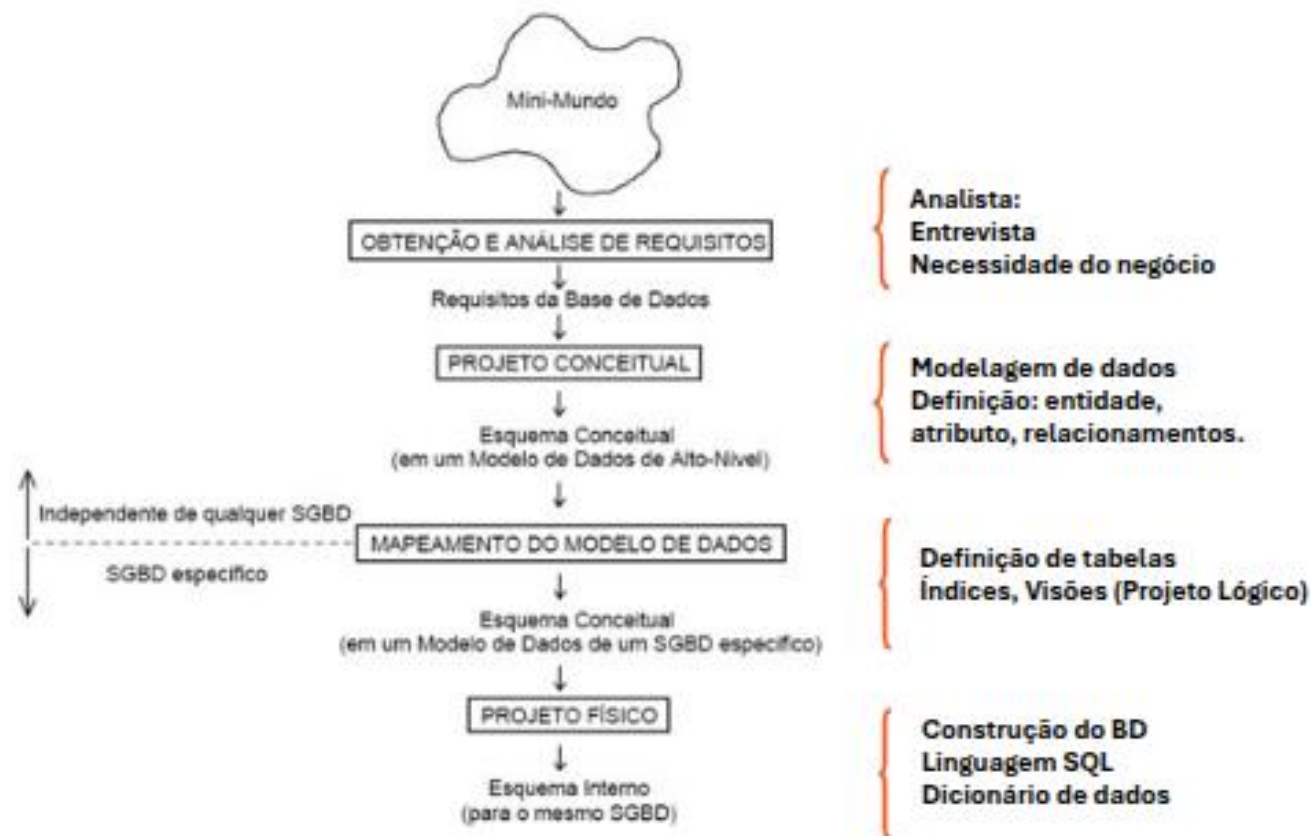


Banco de Dados

Prof^a Angelica Nakagawa Lima
angelica.lima@ibta.edu.br

Revisão



Linguagem SQL

Linguagem SQL

Trata-se de uma linguagem declarativa que tem como função criar, manter, manipular e recuperar dados do banco de dados, bem como administrar e manter a segurança dos dados.

Atualmente é a mais utilizada em Banco de Dados Relacional

Linguagem SQL

Se caracteriza pela utilização de palavras-chaves que são classificadas conforme sua função:

- **DML – Linguagem de Manipulação de Dados:** subconjunto da linguagem usado para inserir, atualizar e apagar dados. Ex.: INSERT, UPDATE, DELETE;
- **DDL – Linguagem de Definição de Dados:** usada para definir novas tabelas e elementos associados. Ex.: CREATE, DROP;
- **DQL – Linguagem de Consulta de Dados:** utilizada para especificar uma consulta (query) como uma descrição do resultado desejado. Ex.: SELECT.

Linguagem SQL

- **DCL – Linguagem de Controle de Dados:** controla os acessos de visualização e manipulação dos dados no banco de dados. Ex.: GRANT, REVOKE;
- **DTL – Linguagem de Transação de Dados:** usado para o controle de transações no banco de dados. Ex.: START TRANSACTION;

Linguagem SQL e SGDBs

- Exemplos de Sistemas Gerenciadores que utilizam SQL:
 - **Oracle (pago);**
 - SQL Server (pago);
 - PostgreSQL (código aberto);
 - Firebird (código aberto);
 - MySQL (código aberto e produto da Oracle);

Linguagem SQL - DDL

- Iremos começar com o grupo de comandos SQL chamados de DDL, pois permitem a definição dos dados.
- Na definição de nomes de tabelas e colunas (atributos) devemos seguir algumas regras:
 - Eles devem conter de 1-30 caracteres, sendo o primeiro alfabético.
 - Eles devem conter apenas caracteres de a-z, A-Z, 0-9, _, \$ e #.
 - Os nomes não podem ser iguais as palavras reservadas do SGBD que estiver sendo utilizado.

Linguagem SQL - DML

- Para criar as colunas da tabela precisamos definir três coisas:
 - Qual o tipo de dado que será armazenado nela;
 - Tamanho;
 - E as restrições.

Linguagem SQL – Tipos de Dados

- **CHAR(n)**: Campo fixo do tipo caractere com tamanho máximo de 2000 bytes.
- **DATE**: Permite data entre 1 de janeiro de 4712 AC até 31 de dezembro de 4712 DC
- **VARCHAR2(n)**: Campo do tipo caractere com tamanho variável e limitado a 4000 bytes.
- **LONG**: Conjunto de caracteres de tamanho variável com até 2 gigabytes.
- **NUMBER(n,d)**: Onde n é o número de dígitos e d o número de casas decimais.

Linguagem SQL – Tipos de Dados

- **RAW(comprimento):** dados binários de comprimento variável. O comprimento máximo é de 255 bytes.
- **LONG RAW:** dados binários com comprimento variável e de tamanho máximo igual a 2 gigabytes.
- **ROWID:** string hexadecimal que representa o endereço único de uma linha numa tabela.
- **BLOB, CLOB, NCLOB e BFILE:** tipos de dados para conteúdos binários até 4 Gigabytes internos ou externos (BFILE) à base de dados.
- **NVARCHAR2, MLSLABEL e NCHAR:** Outros tipos de dados possíveis mas menos usados.

Linguagem SQL – Restrição (constraint)

- São regras estabelecidas para o preenchimento de uma ou mais colunas da tabela. Elas são definidas ao final da especificação de cada coluna ou ao final do comando. Entre as restrições encontram-se:
 - Chaves primárias;
 - Chaves únicas;
 - Chaves estrangeiras;
 - Identificadores de campos obrigatórios;
 - Condições para valores permitidos para determinado campo.

Linguagem SQL - Restrição

- *Constraints* de integridade de dados:

CONSTRAINT	DESCRIÇÃO
NOT NULL	Especifica que esta coluna não pode conter valores nulos.
UNIQUE	Especifica uma coluna ou combinação de colunas que terão seus valores únicos na tabela.
PRIMARY KEY	Identifica a unicidade de cada linha na tabela.
FOREIGN KEY REFERENCES	Estabelece um relacionamento entre a chave estrangeira e a chave primária da tabela referenciada.
CHECK	Especifica uma condição que deve ser verdadeira obedecendo uma regra do negócio.

Instalação e configuração

O que é MySQL?

MySQL é um Sistema de Gerenciamento de Banco de Dados Relacional (SGBD).

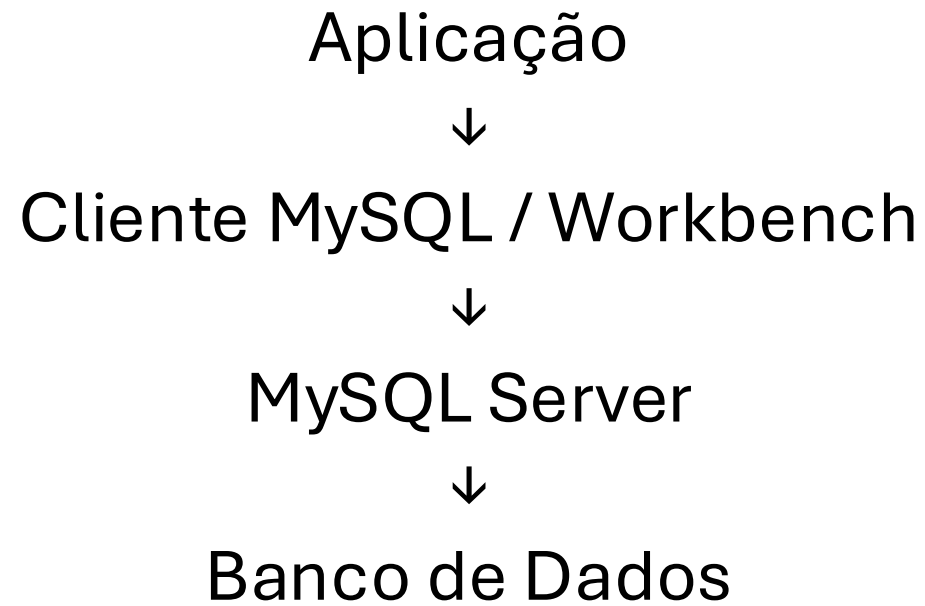
Características:

- Open source
- Amplamente usado na indústria
- Utiliza linguagem SQL
- Alta confiabilidade
- Compatível com Linux, Windows e Mac

Usos comuns:

- Sistemas web
- E-commerce
- Aplicações empresariais
- Ciência de dados

Arquitetura Básica



Arquitetura Básica

O servidor MySQL:

- Armazena dados
- Processa consultas
- Gerencia usuários
- Controla acesso aos dados

Instalação do MySQL Server

Atualizar sistema:

```
sudo apt update  
sudo apt upgrade
```

Instalar MySQL:

```
sudo apt install mysql-server
```

Pacotes instalados:

- MySQL Server
- Cliente MySQL
- Bibliotecas necessárias

Verificando o Serviço

Verificar se o MySQL está rodando:

```
sudo systemctl status mysql
```

Resultado esperado:

```
Active: active (running)
```

Se necessário iniciar:

```
sudo systemctl start mysql  
sudo systemctl enable mysql
```

Configuração de Segurança

Executar configuração inicial:

```
sudo mysql_secure_installation
```

Essa ferramenta permite:

- remover usuários anônimos
- desabilitar login remoto do root
- remover banco de teste
- aplicar regras de segurança

Criando um Usuário

Entrar no MySQL:

```
sudo mysql
```

Criar usuário:

```
CREATE USER 'aluno'@'localhost' IDENTIFIED BY 'aluno123';  
GRANT ALL PRIVILEGES ON *.* TO 'aluno'@'localhost';  
FLUSH PRIVILEGES;
```

Boa prática: não usar root em aplicações.

Interface Gráfica – MySQL Workbench

Instalar Workbench:

```
sudo apt install mysql-workbench
```

Permite:

- executar consultas SQL
- criar tabelas
- visualizar bancos
- modelar banco de dados

Criando uma Conexão

No MySQL Workbench:

Connection Name: MySQL Local

Hostname: localhost

Port: 3306

Username: aluno

Clique em 'Test Connection' para validar.

Testando o Banco

```
CREATE DATABASE aula_bd;
```

```
USE aula_bd;
```

```
CREATE TABLE alunos (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(100)  
);
```


Linguagem SQL

Linguagem SQL – Criação de Tabela

```
CREATE TABLE nome_tabela  
( nome_coluna tipo de dado | constraint_tabela ,  
nome_coluna tipo de dado | constraint_tabela )
```

- **Nome_tabela:** é o nome da tabela
- **Nome_coluna:** é o nome da coluna
- **Tipo de dado:** é o tipo de dado da coluna
- **Constraint_tabela:** é a “constraint” ou restrição para a coluna.

Linguagem SQL – Criação de tabela

□ Exemplo:

```
CREATE TABLE Fornecedor
( Cd_Fornecedor      Number (4),
  Nm_Fornecedor      Varchar2 (50),
  Ds_Endereco         Varchar2 (70),
  Nr_Fone             Number (12),
  Dt_Nascimento       Date,
  le_Sexo             Char (1),
  le_Fisica_Juridico  Char (1),
  Cd_Situacao         Number (3)
)
```

Linguagem SQL – Criação de tabela

- **Valores Default:** para definir um valor padrão, devemos colocar, após o tipo e tamanho do campo, a palavra **DEFAULT** seguida de uma expressão, que será o valor padrão para o campo.

```
CREATE TABLE Nota_Fiscal
( Id_Nota      Number (4),
  VI_Nota      Number (5,2),
  Dt_Emissao   Date Default Sysdate,
  VI_imposto   Number (5,2)
)
```

Linguagem SQL – Criação de tabela

- **Constraint Primary Key:** Especifica uma ou mais colunas que compõem a chave primária de uma tabela.

```
CREATE TABLE Cliente
(cd_cliente    number (4) primary key,
 nm_cliente   varchar2 (50),
 ds_endereco  varchar2 (70),
 cd_municipio number (5),
 sg_estado    char (2),
 nr_cep       varchar2 (8),
 nr_ddd       number (3),
 nr_fone      number(7),
 ie_sexo      char(1)
)
```

OBS: a expressão primary key declarada ao lado do campo, somente é válida para chaves primárias simples

Linguagem SQL – Criação de Tabela

- **Nomeando a Primary Key:** seguiremos o seguinte padrão:

constraint **nome_tabela_nome_campo_tipodaconstraint**

```
CREATE TABLE Cliente
(cd_cliente      number (4)
    constraint cliente_cd_cliente_pk primary key,
nm_cliente      varchar2 (50),
ds_endereco     varchar2 (70),
cd_municipio    number (5),
sg_estado       char (2),
nr_cep          varchar2 (8),
nr_ddd          number (3),
nr_fone         number(7),
ie_sexo         char(1)
)
```

DROP TABLE CLIENTE

Linguagem SQL – Criação de Tabela

- Uma forma mais organizada...

```
CREATE TABLE Cliente
(cd_cliente    number (4),
 nm_cliente   varchar2 (50),
 ds_endereco  varchar2 (70),
 cd_municipio number (5),
 sg_estado    char (2),
 nr_cep        varchar2 (8),
 nr_ddd        number (3),
 nr_fone       number(7),
 ie_sexo       char(1),
              constraint cliente_cd_cliente_pk primary key (cd_cliente)
)
```

Linguagem SQL – Criação de Tabela

- ***Constraint* Primary Key composta:**

```
CREATE TABLE Historico
(cd_cliente  number (4),
 dt_compra   date,
 vl_compra   number (12,2),
  constraint historico_pk primary key (cd_cliente, dt_compra)
)
```


Linguagem SQL – Criação de Tabela

- **Constraint Unique:** Define uma ou mais colunas que não podem ter valor repetido em mais de uma linha da tabela.
- Por exemplo, não existem duas pessoas com o mesmo CPF ou número do PIS, mas estes campos não serão colocados como chave primária.

```
CREATE TABLE Estado
(sg_estado      char(2) primary key,
nm_estado      varchar2 (35),
               constraint Estado_nm_estado_UN UNIQUE (nm_estado)
)
```

Linguagem SQL – Criação de Tabela

- **Constraint Foreign Key:** Referencia um atributo que é chave primária de outra tabela com o propósito de implementar o relacionamento entre as tabelas.

```
CREATE TABLE Cliente
(cd_cliente      number (4),
 nm_cliente      varchar2 (50),
 ds_endereco     varchar2 (70),
 cd_municipio    number (5),
 sg_estado       char (2),
 nr_cep          varchar2 (8),
 nr_ddd          number (3),
 nr_fone         number(7),
 constraint cliente_cd_cliente_pk primary key (cd_cliente),
 constraint cliente_sg_estado_fk foreign key (sg_estado)
 references Estado(sg_estado)
)
```

Linguagem SQL – Criação de Tabela

□ ***Constraint Foreign Key (cont.):***

- Caso o tipo de dados da coluna na tabela inicial e na tabela referenciada sejam diferentes, será apresentado um erro, pois tem que ser o mesmo tipo;
- O uso de chaves estrangeiras garante que não existirão linhas órfãs nas tabelas-filhas (tabelas que possuem dados que devem estar cadastrado previamente em outra tabela, denominada tabela mãe).

Linguagem SQL – Criação de Tabela

- **Constraint Check:** Define um conjunto de valores permitidos ou condição para inserção de valores em uma determinada coluna.

```
CREATE TABLE Cliente
(cd_cliente      number (4),
 nm_cliente      varchar2 (50),
 ds_endereco     varchar2 (70),
 cd_municipio    number (5),
 sg_estado       char (2),
 ...,
 ie_sexo         char(1),
 constraint cliente_cd_cliente_pk primary key (cd_cliente),
 constraint cliente_sg_estado_fk foreign key (sg_estado)
 references Estado(sg_estado),
 constraint cliente_ie_sexo_ck check(ie_sexo in ('F', 'M'))
)
```

Linguagem SQL – Criação de Tabela

- **Constraint Not Null:** Indica que a inserção de algum valor é obrigatória na coluna. Somente pode ser declarado junto à coluna e não recebe nome de

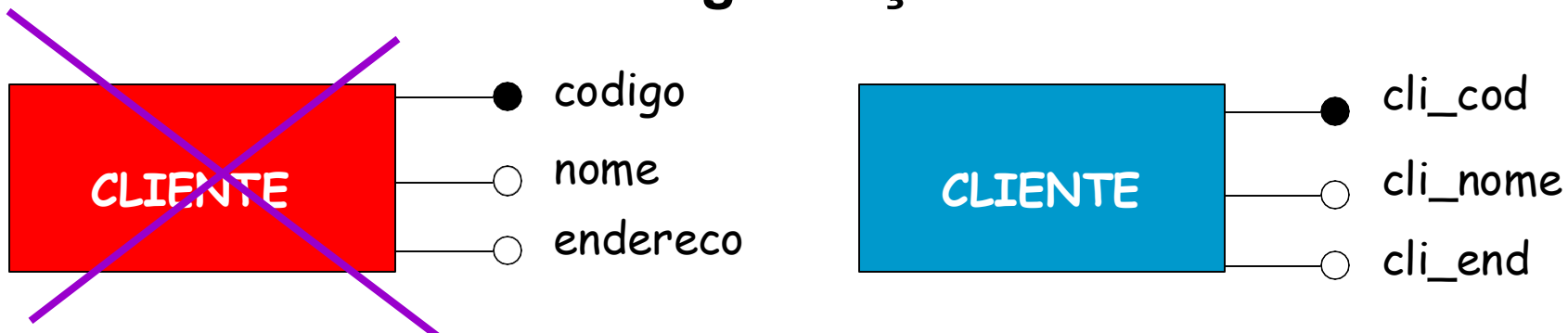
```
CREATE TABLE Cliente
(cd_cliente      number (4),
 nm_cliente     varchar2 (50) not null,
 ds_endereco    varchar2 (70) not null,
 cd_municipio   number (5),
 sg_estado      char (2),
 ...,
 ie_sexo        char(1),
 constraint cliente_cd_cliente_pk primary key (cd_cliente),
 constraint cliente_sg_estado_fk foreign key (sg_estado)
 references Estado(sg_estado)
 constraint cliente_ie_sexo_ck check(ie_sexo in ('F', 'M'))
)
```

Linguagem SQL - Padronização

- **Padronização:** a padronização em nomes de campos e restrições de tabelas é muito importante para facilitar o entendimento e a velocidade de desenvolvimento por programadores e analistas.
- Apresentaremos 2 técnicas de padrões utilizados em banco de dados:
 - **Trigramação**
 - **Qualificador _nome descritivo**

Linguagem SQL - Padronização

- ❑ **Trigramação:** um **trigrama** é uma cadeia de caracteres normalmente constituída pelas três primeiras letras ou pelas três letras mais significativas de uma Entidade, que são utilizadas como prefixo dos atributos ou nomes de campos de uma Tabela.
- ❑ Para escolher o nome mais adequado de um atributo, sugere-se a adoção de uma notação padronizada, utilizando a técnica de **Trigramação**.



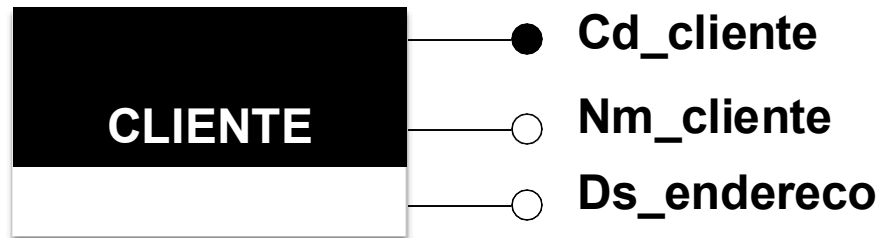
Linguagem SQL - Padronização

- **Qualificador _nome descritivo**
 - os qualificadores mais utilizados, com os seus devidos significados, são apresentados na tabela ao lado

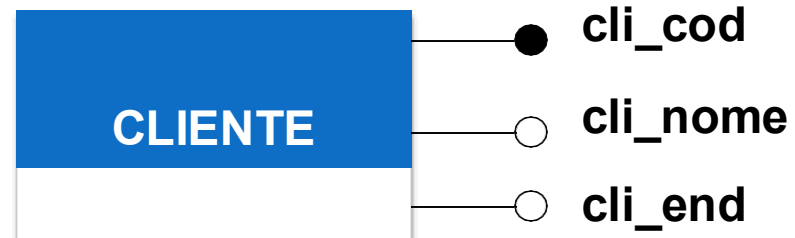
Qualificador	Significado
Cd	Código
Nm	Nome
Nr	Número
VI	Valor
Qt	Quantidade
Tx	Taxa ou percentual
Ds	Descrição
Sg	Sigla
Dt	Data
Hr	Hora
Ie ou Id	Identificador
Im	Imagem

Linguagem SQL - Padronização

Qualificador_nome descritivo



Trigramação

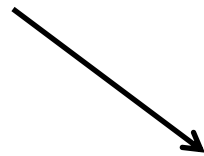


Linguagem SQL - Alteração de tabelas

- Após a criação da estrutura de uma tabela, se necessário, podemos incluir (ADD), excluir (DROP) ou modificar (MODIFY) colunas ou *constraints* da tabela. Para isso utilizamos o comando ALTER.
- Exemplo:

ALTER TABLE **Cliente**

ADD (**ie_fisica_juridica** **CHAR(1)**)



Incluindo uma **nova coluna**

Linguagem SQL - Alteração de tabelas

□ Outros exemplos:

ALTER TABLE **Cliente**

MODIFY (**nm_cliente** **NOT NULL**)



Alterando a **obrigatoriedade** da coluna

ALTER TABLE **Cliente**

MODIFY (**ds_endereco** **NUMBER(3)**)



Alterando o **tipo** da coluna

Linguagem SQL - Alteração de tabelas

```
ALTER TABLE Cliente  
MODIFY (nr_cep VARCHAR2(12))
```



Alterando o **tamanho** da coluna

```
ALTER TABLE Fornecedor  
ADD (CONSTRAINT fornecedor_ie_fis_jur_ck  
CHECK (ie_fisica_juridico IN('F','J')));
```



Acrescentando **restrições** a uma tabela

Linguagem SQL - Alteração de tabelas

ALTER TABLE Fornecedor

DISABLE CONSTRAINT fornecedor_ie_fis_jur_ck;



Desabilitando **restrições** a uma tabela

ALTER TABLE Fornecedor

DROP CONSTRAINT fornecedor_ie_fis_jur_ck;



Excluindo **restrições** a uma tabela

Linguagem SQL - Alteração de tabelas

```
ALTER TABLE Fornecedor  
ADD (CONSTRAINT fornecedor_cd_fornecedor_pk  
PRIMARY KEY (cd_fornecedor));
```



Definindo a **chave primária** na
tabela

```
ALTER TABLE Fornecedor  
DROP (ie_fisica_juridica);
```



Apagando uma **coluna** na
tabela

Linguagem SQL - Visualizando Definições

- Como já vimos, é possível visualizar as definições estruturais da tabela usando o comando DESC.
Exemplo:
DESC cliente;
- Também podemos consultar as informações nas tabelas de controle **USER_CONSTRAINTS** e **USER_CONS_COLUMNS**.

Linguagem SQL - Visualizando Definições

```
SELECT * FROM user_constraints  
WHERE table_name='cliente';
```

```
SELECT constraint_name, constraint_type  
FROM user_constraints WHERE table_name='cliente';
```

```
SELECT * FROM cat;
```



Lista todas as tabelas do
banco

Linguagem SQL – Excluir tabela

- A exclusão é feita usando o comando DROP.
- Exemplo:

DROP TABLE **cliente**;

- Nas exclusões de tabelas não podemos esquecer das dependências. Nestes casos, a tabela dependente tem que ser excluída primeiro.
- No caso da criação das tabelas é o contrário, primeiro cria-se a tabela mãe e depois a dependente

Agradecimentos

Slides baseados nas aulas da prof. Cristiane - Unicsul